

## Prompts:

Build a full-stack calculator application with a React frontend and a backend microservice. The frontend should consume the backend API to perform basic and advanced arithmetic operations. Focus on clean design, maintainable code, and testable architecture.

**Requirements**

**Functional**

**Operations:**

Addition, Subtraction, Multiplication, Division

Optional: Exponentiation, Square Root, Percentage

**Frontend (React):**

Intuitive UI for entering input and displaying results

Input validation and error handling

Responsive design (basic mobile support)

**Backend (REST API):**

Expose endpoints for calculator operations

Validate input and handle edge cases (division by zero, invalid data)

Return results in JSON format

**Non-Functional**

Clean, readable, and idiomatic code (frontend and backend)

Unit tests covering key functionality for both layers

**Documentation:** setup instructions, API usage, and design rationale

Worked for 55s >

I've created the implementation plan. Here's a quick summary of the key decisions:

- **Monorepo** with `client/` (React + Vite) and `server/` (Express.js) directories
- **Single API endpoint** `POST /api/calculate` that dispatches to a pure-function service layer — keeps the backend clean and easily testable
- **7 operations:** add, subtract, multiply, divide, power, sqrt, percentage
- **Glassmorphism dark UI** with gradient accents, Inter font, and micro-animations
- **Vitest** for both layers (unified tooling), **supertest** for API integration tests
- **Docker Compose** for single-command deployment

Please review the plan and let me know if you'd like any changes before I start building.

📄 **Implementation Plan**



Puedes ajustar el plan de implementación para que el backend se cree utilizando "Go" (golang) también considera que ambos microservicios (back y front) se puedan ejecutar de forma separada en contenedores docker, e incluye una opción con docker compose para poder ejecutar ambos servicios en conjunto

Worked for 53s >

Plan actualizado con los siguientes cambios clave:

- **Backend en Go** — `net/http` + `chi` router, estructura idiomatic con `internal/` (handler, service, model, validator), tests con `stdlib` `testing` + `httpptest`
- **Dockerfiles independientes** — cada microservicio tiene su propio Dockerfile multi-stage y puede ejecutarse por separado
- **docker-compose** — orquesta ambos servicios en una red compartida con un solo comando
- **3 modos de ejecución** documentados: desarrollo local, contenedores individuales, y full stack con compose

Revisa el plan y apruébalo para comenzar la implementación.

 **Implementation Plan**



ok, procede con la implementación

puedes agregar una respuesta "default" a la petición en `http://localhost:8080`, si hago una petición `get` a esa url por default regresa un 404, puedes regresar la lista de endpoints disponibles

Worked for 2m >

✅ 25/25 tests siguen pasando.

Ahora `GET http://localhost:8080/` regresará:

```
json
{
  "service": "Calculator API",
  "version": "1.0.0",
  "endpoints": [
    {
      "method": "GET",
      "path": "/",
      "description": "API discovery - lists all available endpoints"
    },
    {
      "method": "GET",
      "path": "/api/health",
      "description": "Health check - returns service status and timestamp"
    },
    {
      "method": "POST",
      "path": "/api/calculate",
      "description": "Perform a calculation. Body: {\"operation\": \"add|subtract|multiply|divide|power|sqrt|percentage\", \"a\": number, \"b\": number}"
    }
  ]
}
```

Los cambios fueron en `handler/calculator.go` (nuevo handler `Root`) y `main.go:L44` (registro de la ruta).

también modifica los paths para que sean "api/v1/health" y "/api/v1/calculate"

Worked for 55s >

✅ **Todo pasa — 54/54 tests.**

| Suite          | Tests | Estado |
|----------------|-------|--------|
| Backend Go     | 25    | ✅ PASS |
| Frontend React | 29    | ✅ PASS |

Los paths actualizados son:

- `GET /` → discovery de endpoints
- `GET /api/v1/health` → health check
- `POST /api/v1/calculate` → operaciones

5 files changed +7 -7 >

 Review

  

Cómo estamos manejando multiples ambientes en los proyectos?

Worked for 34s >

Aquí está el panorama actual — es bastante **mínimo**:

### Estado actual del manejo de ambientes

| Servicio | Variable | Dónde                       | Qué hace                                          |
|----------|----------|-----------------------------|---------------------------------------------------|
| Backend  | PORT     | <code>go main.go:L19</code> | Puerto del servidor (default: <code>8080</code> ) |

100% 100% 100% 100% 100% 100%

100% 100% 100% 100% 100% 100%

## Lo que falta

Siendo honesto, el manejo de ambientes es muy básico. No tenemos:

1. **Archivos `.env`** por ambiente ( `.env.development` , `.env.production` , `.env.staging` )
2. **CORS configurable** — los origins están hardcoded a `localhost:3000` y `localhost:5173` en [main.go:L36](#)
3. **Log level** configurable — siempre usa el middleware Logger de chi
4. **Perfiles de docker-compose** — no hay `docker-compose.prod.yml` o `docker-compose.dev.yml`
5. **Variables de entorno en docker-compose** — no se pasan `PORT` , `VITE_API_URL` , ni CORS origins al contenedor

En resumen: el backend solo lee `PORT` y el frontend solo lee `VITE_API_URL` , todo lo demás está hardcoded. ¿Te gustaría que implemente un manejo de ambientes más robusto con archivos `.env` y configuración por perfil?

12:30 PM



Sí, implementa un manejo de ambientes más robusto con configuración por perfil y con variables de entorno

Worked for 2m >

Worked for 1s >

veo que en el archivo vite.config.js la url está hardcoded:  
http://localhost:8080  
No debería tomarla de los archivos de configuración?

Worked for 31s >

Toda la razón, excelente observación. Acabo de hacer ese cambio para que el `target` del proxy también provenga de las variables de entorno.

Modifiqué `vite.config.js` para usar la función `loadEnv` que nos expone Vite. Ahora puedes definir el destino configurando la variable `VITE_PROXY_TARGET` en los archivos de ambiente.

Por ejemplo, agregué esto a `.env.development`:

```
env
# Target for local API proxy (used by vite.config.js)
VITE_PROXY_TARGET=http://localhost:8080
```

Así evitamos depender del valor hardcoded. La aplicación compila correctamente después del cambio.

3 files changed +28 -16 >

 Review

12:43 PM

  

también agrega que se busque la variable de entorno para el port que expone la app web: `CALCULATOR_FRONTEND_PORT` por default sería el 3000

Worked for 16s >

Listo. Actualicé `vite.config.js` para que el puerto se asigne de forma dinámica.

El bloque del servidor en la configuración de Vite ahora se ve así:

```
javascript
server: {
  port: parseInt(env.CALCULATOR_FRONTEND_PORT || '3000', 10),
  strictPort: false,
  proxy: {
```

Y también agregué la variable a `.env.example` para que quede documentada:

```
env
# Frontend development server port
CALCULATOR_FRONTEND_PORT=3000
```

Con esto, si inicias tu aplicación con `npm run dev` y tienes la variable de entorno `CALCULATOR_FRONTEND_PORT` definida (por ejemplo exportada en tu terminal a `4000`), el servidor web local se levantará en el puerto 4000; de lo contrario usará el puerto 3000 por defecto.

2 files changed +5 -1 >

Review

## Cómo puedo obtener un reporte del code coverage de los tests?

Worked for 5m >

Para obtener un reporte de cobertura (code coverage) de los tests en ambos proyectos, he configurado las herramientas necesarias. Aquí te muestro cómo hacerlo y cuáles son los resultados actuales.

### 1. Frontend (React / Vite)

He instalado `@vitejs/coverage` y agregado el script a tu `package.json`. Para generar el reporte, entra a la carpeta `client/` y ejecuta:

```
bash
npm run test:coverage
```

#### Resultado actual (Frontend): ~81% Global

| Archivo / Componente       | Statements   | Branch       | Uncovered        |
|----------------------------|--------------|--------------|------------------|
| <b>utils/validation.js</b> | 100%         | 100%         | -                |
| <b>services/api.js</b>     | 100%         | 84.6%        | Línea 45         |
| <b>Display.jsx</b>         | 100%         | 85.7%        | Línea 37         |
| <b>Button.jsx</b>          | 80.7%        | 100%         | Líneas 24-28     |
| <b>Calculator.jsx</b>      | 76.2%        | 70%          | Múltiples líneas |
| <b>Total Global</b>        | <b>81.3%</b> | <b>78.6%</b> |                  |

(Vite generará adicionalmente una carpeta `coverage/` que incluye un reporte en HTML que puedes abrir en tu navegador para ver exactamente qué líneas faltan).



Ambos proyectos tienen una excelente base de cobertura (81% y casi 100%). ¿Te gustaría que agreguemos pruebas específicas para cubrir las líneas faltantes en el componente

`Calculator.jsx`?

3 files changed +8 -8 >

 Review

  

Sí, agrega tests para tener mayor cobertura

1:03 PM  

Worked for 4m >

Timed 15 seconds >

Verify frontend coverage finished

¡Listo! He agregado nuevos tests en ambos proyectos para incrementar sustancialmente el porcentaje de cobertura.

## Backend (Go)

He creado el archivo `validator_test.go` para cubrir de manera explícita y exhaustiva todas las posibles validaciones de la petición (operaciones faltantes, divisiones entre cero, números `NaN` o `Infinito`, etc.).

- `internal/service`: 100%
- `internal/config`: 100%
- `internal/handler`: 94.7%
- `internal/validator`: Subió de 0% (solo testeado vía el handler) a **92.9%**

## Frontend (React / Vite)

He agregado tests específicos en `Calculator.test.jsx`, `Display.test.jsx`, y he mejorado `api.test.js` logrando los siguientes incrementos:

- **Cobertura Global**: Subió del 81% a **87.2%**.
- `utils/validation.js`: 100%

Agrega en el archivo README.md en la sección de "Testing" los comandos para ejecutar las pruebas unitarias y los comandos para obtener la cobertura de los tests

Worked for 18s >

¡Hecho! He actualizado la sección de `Testing` en el archivo `README.md` agregando tanto los comandos para ejecutar las pruebas, como los comandos para obtener sus respectivos reportes de cobertura (coverage).

Aquí tienes un resumen de lo que quedó documentado:

#### Para el Frontend:

- `npm run test:watch` para pruebas iterativas mientras desarrollas.
- `npm test` para una ejecución única.
- `npm run test:coverage` para sacar el reporte detallado con soporte de Vitest y V8.

#### Para el Backend:

- `go test ./... -v` para pruebas regulares.
- `go test -coverprofile=coverage.out ./...` seguido de `go tool cover -html=coverage.out` para ver el reporte interactivo directo en tu navegador.

¿Hay alguna otra sección del README o del código que te gustaría documentar o ajustar?

1 file changed +23 -0 >

 Review

1:11 PM

  

agrega un archivo `.gitignore` para omitir subir todo lo innecesario

Worked for 14s >

He creado el archivo `.gitignore` en la raíz del proyecto. He configurado las reglas para ignorar:

1. **Frontend:** Las carpetas `node_modules/`, `dist/` (builds compilados) y los reportes de `coverage/`.
2. **Backend:** Los binarios compilados de Go (`.exe`, `.out`), así como los archivos generados por las herramientas de testing (`coverage.out`).
3. **Generales:** Archivos basura del sistema operativo (`.DS_Store`), logs (`*.log`) y carpetas locales de editores de código (`.vscode/`, `.idea/`).
4. **Variables de Entorno:** Se ignorarán los archivos `.env` y `.env.local` puros (que normalmente contienen contraseñas o secretos de producción que no se deben subir), pero permitiremos que suban nuestros `.env.development`, `.env.staging` y `.env.production` ya que actualmente funcionan como plantillas de configuración del comportamiento en la aplicación de Vite.

De esta manera, si inicializas un repositorio (con `git init`), solo se le dará seguimiento al código fuente limpio. ¿Hay algo más en lo que te pueda ayudar con este proyecto?

1 file changed +44 -0 >

 Review

1:23 PM

  

puedes hacer el front responsivo, para que se adapta a diferentes tamaños de pantallas

Worked for 38s >

¡Hecho! He mejorado la responsividad del frontend haciendo modificaciones en `App.css` e `index.css`.

Para pantallas grandes (Tablets, Escritorio), la calculadora mantendrá su diseño original: una tarjeta de cristal (glassmorphism) elegante y centrada con un ancho máximo de 400px.

Sin embargo, para pantallas móviles o ventanas pequeñas (menos de 480px de ancho) he añadido reglas especiales para darle un **"look" de aplicación móvil nativa**:

1. **Ocupa el 100% de la pantalla:** Eliminé los márgenes y bordes redondeados externos para que la calculadora abarque toda la altura y anchura del dispositivo.
2. **Flexbox Dinámico:** La pantalla donde aparecen los números ahora "empuja" dinámicamente los botones hacia el fondo, aprovechando todo el espacio sobrante en la parte superior.
3. **Botones Amigables para Touch:** Incrementé la altura de los botones a 72px para que sean más fáciles y precisos de tocar en pantallas táctiles.
4. **Soporte para "Notch":** Agregué espacio dinámico (mediante `env(safe-area-inset-bottom)`) para evitar que los controles del iPhone u otros teléfonos modernos se superpongan con la última fila de botones.
5. **Ajuste Proporcional:** En teléfonos extremadamente delgados (menos de 360px), las fuentes y los botones escalarán ligeramente hacia abajo de manera automática para que no se desborden de la pantalla.

Si abres tu proyecto en el navegador y empiezas a encoger la ventana, notarás inmediatamente cómo el diseño pasa de ser una "tarjeta" a una vista inmersiva completa en cuanto la ventana se hace pequeña.

¿Qué te gustaría hacer a continuación? Podemos levantar y probar ambos proyectos (frontend y backend) con Docker si gustas.

2 files changed +48 -27 >

 Review